COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	September 23, 2026	**DATE SUBMITTED:**	September 23, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

Activity 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

Task 1:

task1.cpp > main()

```cpp
#include <iostream>

template <typename T>
class treeNode {
public:
    T data;
    treeNode<T>* firstChild;   // Pointer to the first child
    treeNode<T>* nextSibling;  // Pointer to the next sibling

    treeNode(T val) {
        data = val;
        firstChild = nullptr;
        nextSibling = nullptr;
    }

    // Function to attach a child node
    void addChild(treeNode<T>* child) {
        if (firstChild == nullptr) {
            firstChild = child;
        } else {
            treeNode<T>* temp = firstChild;
            while (temp->nextSibling != nullptr) {
                temp = temp->nextSibling;
            }
            temp->nextSibling = child;
        }
    }
};

```

Observations: The code defines a generic treeNode template class using the first-child/next-sibling representation. The addChild() function checks whether firstChild is null to attach the initial child node; otherwise, it traverses the sibling linked list via nextSibling to append the new node at the end of the sibling chain.

```
29
30  int main() {
31
32      treeNode<char>* A = new treeNode<char>('A');
33
34      // Level 1: Children of A
35      treeNode<char>* B = new treeNode<char>('B');
36      treeNode<char>* C = new treeNode<char>('C');
37      treeNode<char>* D = new treeNode<char>('D');
38      treeNode<char>* E = new treeNode<char>('E');
39      treeNode<char>* F = new treeNode<char>('F');
40      treeNode<char>* G = new treeNode<char>('G');
41
42      A->addChild(B);
43      A->addChild(C);
44      A->addChild(D);
45      A->addChild(E);
46      A->addChild(F);
47      A->addChild(G);
48
49      treeNode<char>* H = new treeNode<char>('H');
50      D->addChild(H);
51
52      treeNode<char>* I = new treeNode<char>('I');
53      treeNode<char>* J = new treeNode<char>('J');
54      E->addChild(I);
55      E->addChild(J);
56
```

Observations: The main() function constructs a general tree hierarchical structure using dynamic memory allocation (new operator). Root node 'A' is instantiated and populated with children 'B' through 'G', while lower-level child relationships are established for nodes 'D' (attaching 'H') and 'E' (attaching 'I' and 'J')

```
57      treeNode<char>* K = new treeNode<char>('K');
58      treeNode<char>* L = new treeNode<char>('L');
59      treeNode<char>* M = new treeNode<char>('M');
60      F->addChild(K);
61      F->addChild(L);
62      F->addChild(M);
63
64      treeNode<char>* N = new treeNode<char>('N');
65      G->addChild(N);
66
67      treeNode<char>* P = new treeNode<char>('P');
68      treeNode<char>* Q = new treeNode<char>('Q');
69      J->addChild(P);
70      J->addChild(Q);
71
72      std::cout << "General Tree initialized successfully.\n";
73
74      return 0;
75  }
```

The construction of the tree is completed by attaching children 'K', 'L', and 'M' to parent node 'F', child 'N' to node 'G', and deepest leaf nodes 'P' and 'Q' to node 'J'. Once all pointer links are established across the multi-level hierarchy, a success message is printed to confirm full tree initialization.

Output:

```
General Tree initialized successfully.
```

Observations: The successful printing of "General Tree initialized successfully." validates that all nodes (A through Q) were properly instantiated in dynamic memory without runtime crashes or pointer exceptions. It confirms that the multi-level hierarchy was completely assembled using the First-Child / Next-Sibling linked representation, establishing all parent-child and sibling relationships throughout the tree.

Task 2

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1

E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	2
P	0	3
Q	0	3

Task 3.1

Traversal Strategy	Sequence output
Pre-Order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-Order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A
In Order	B, A, H, D, I, E, P, J, Q, K, F, L, M, N, G

Task 3.2

```cpp
template <typename T>
void preOrder(treeNode<T>* node) {
    if (node == nullptr) return;

    std::cout << node->data << " ";
    treeNode<T>* child = node->firstChild;
    while (child != nullptr) {
        preOrder(child);
        child = child->nextSibling;
    }
}

template <typename T>
void postOrder(treeNode<T>* node) {
    if (node == nullptr) return;

    treeNode<T>* child = node->firstChild;
    while (child != nullptr) {
        postOrder(child);
        child = child->nextSibling;
    }
    std::cout << node->data << " ";
}

```

```cpp
template <typename T>
void inOrder(treeNode<T>* node) {
    if (node == nullptr) return;

    treeNode<T>* child = node->firstChild;
    if (child != nullptr) {
        inOrder(child);
    }

    std::cout << node->data << " ";

    if (child != nullptr) {
        child = child->nextSibling;
        while (child != nullptr) {
            inOrder(child);
            child = child->nextSibling;
        }
    }
}
```

Observations:
The outputs generated by the automated C++ traversal functions match the hand-calculated traversal paths from Task 3.1 precisely. The pre-order function prints parent nodes prior to visiting their subtrees, post-order processes leaf nodes completely before visiting parent nodes, and in-order evaluates the first leftmost child branch before processing the parent node and remaining sibling branches.

Task 3.3

```cpp
template <typename T>
bool searchHelper(treeNode<T>* node, int CHOICE, T KEY) {
    if (node == nullptr) return false;

    if (CHOICE == 1 && node->data == KEY) return true;

    treeNode<T>* child = node->firstChild;

    if (CHOICE == 2) {
        if (child != nullptr && searchHelper(child, CHOICE, KEY)) return true;
        if (node->data == KEY) return true;
        if (child != nullptr) child = child->nextSibling;
    }

    while (child != nullptr) {
        if (searchHelper(child, CHOICE, KEY)) return true;
        child = child->nextSibling;
    }

    if (CHOICE == 3 && node->data == KEY) return true;

    return false;
}

template <typename T>
void findData(treeNode<T>* root, int CHOICE, T KEY) {
    if (searchHelper(root, CHOICE, KEY)) {
        std::cout << "\"" << KEY << " was found!\"\n";
    }
}
```

```cpp
std::cout << "Task 3.3:\n";
findData(A, 1, 'J');
std::cout << "\n";
```

Output:

```
Task 3.3:
"J was found!"
```

Observations:

The outputs generated by the automated C++ traversal functions match the hand-calculated traversal paths from Task 3.1 precisely. The pre-order function prints parent nodes prior to visiting their subtrees, post-order processes leaf nodes completely before visiting parent nodes, and in-order evaluates the first leftmost child branch before processing the parent node and remaining sibling branches.

Task 3.4:

```
template <typename T>
bool BinarySearchTree<T>::search(T value) {
    if (Key == value) {
        return true;
    }
    if (value < Key && left != nullptr) {
        return left->search(value);
    }
    if (value > Key && right != nullptr) {
        return right->search(value);
    }
    return false;
}
```

```
164     std::cout << "Task 3.4:\n";
165     treeNode<char>* O = new treeNode<char>('O');
166     G->addChild(O);
167     findData(A, 1, 'O');
168
```

Output:

```
Task 3.4:
"O was found!"
```

Observations:
Dynamically appending node 'O' as a child of node 'G' extends the tree structure in memory. When findData executes, the recursive traversal successfully navigates down through root 'A', accesses node 'G', locates the newly attached leaf node 'O', and confirms its existence by outputting "O was found!".

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1:

BST.h > ...

```h
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class BinarySearchTree {
private:
    T Key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:
    // Constructor
    BinarySearchTree(T value);

    // Insertion Method
    void insert(T value);

    // Traversal Methods
    void preOrder();
    void inOrder();
    void postOrder();
};

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    Key = value;
    left = nullptr;
    right = nullptr;
}

template <typename T>
void BinarySearchTree<T>::insert(T value) {
```

```
35      if (value < Key) {
36          if (left == nullptr) {
37              left = new BinarySearchTree<T>(value);
38          } else {
39              left->insert(value);
40          }
41      } else if (value > Key) {
42          if (right == nullptr) {
43              right = new BinarySearchTree<T>(value);
44          } else {
45              right->insert(value);
46          }
47      }
48  }
49
50  template <typename T>
51  void BinarySearchTree<T>::preOrder() {
52      std::cout << Key << " ";
53      if (left != nullptr) {
54          left->preOrder();
55      }
56      if (right != nullptr) {
57          right->preOrder();
58      }
59  }
60
61  template <typename T>
62  void BinarySearchTree<T>::inOrder() {
63      if (left != nullptr) {
64          left->inOrder();
65      }
```

```
66      std::cout << Key << " ";
67      if (right != nullptr) {
68          right->inOrder();
69      }
70  }
71
72
73  template <typename T>
74  void BinarySearchTree<T>::postOrder() {
75      if (left != nullptr) {
76          left->postOrder();
77      }
78      if (right != nullptr) {
79          right->postOrder();
80      }
81      std::cout << Key << " ";
82  }
83
84  #endif
```

main.cpp > main()

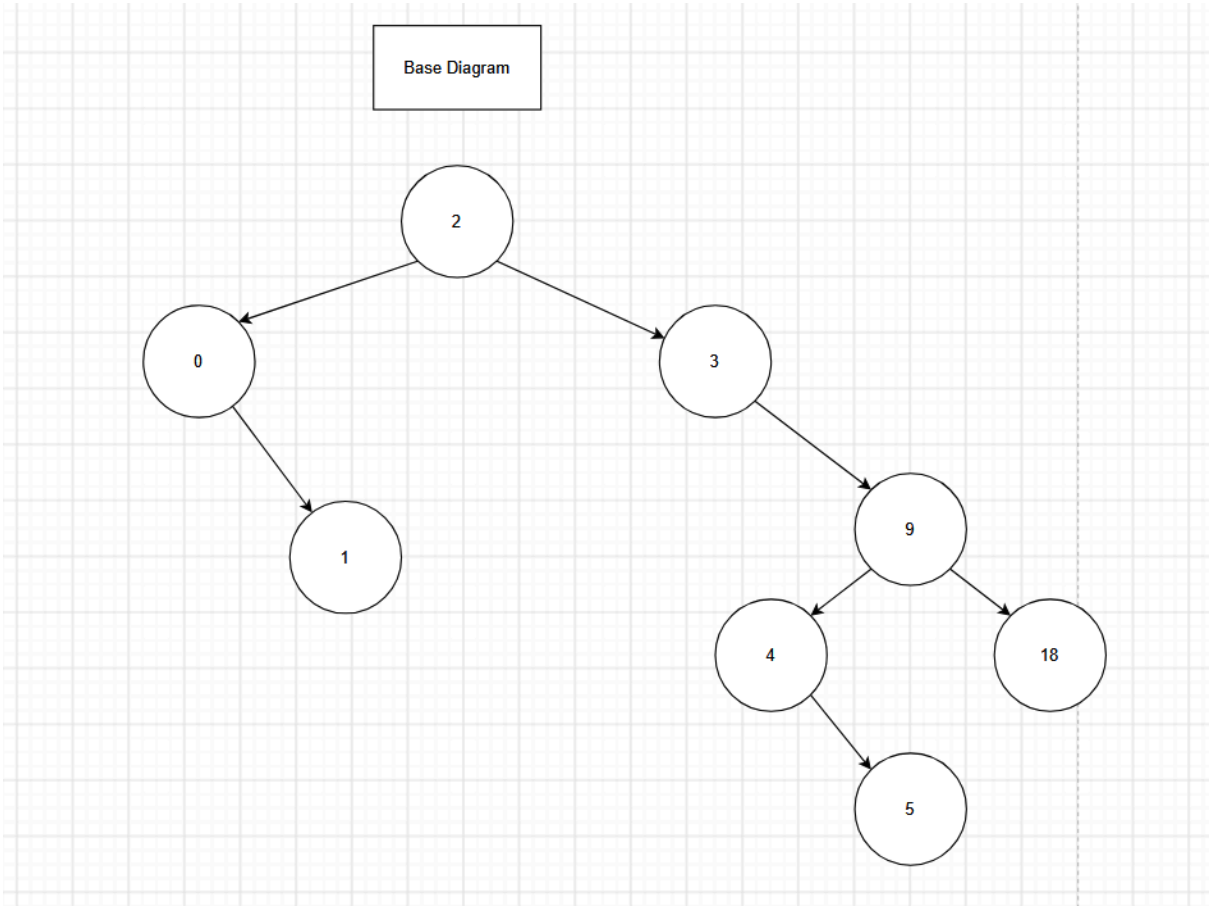
```
1   #include <iostream>
2   #include "BST.h"
3
4   int main() {
5       BinarySearchTree<int> bst(2);
6
7       int sequence[] = {3, 9, 18, 0, 1, 4, 5};
8       int size = sizeof(sequence) / sizeof(sequence[0]);
9
10      std::cout << "Binary Search Tree constructed and populated successfully.\n";
11      std::cout << "Inserted sequence: 2, ";
12      for (int i = 0; i < size; i++) {
13          bst.insert(sequence[i]);
14          std::cout << sequence[i] << (i < size - 1 ? ", " : "\n");
15      }
```

Output:

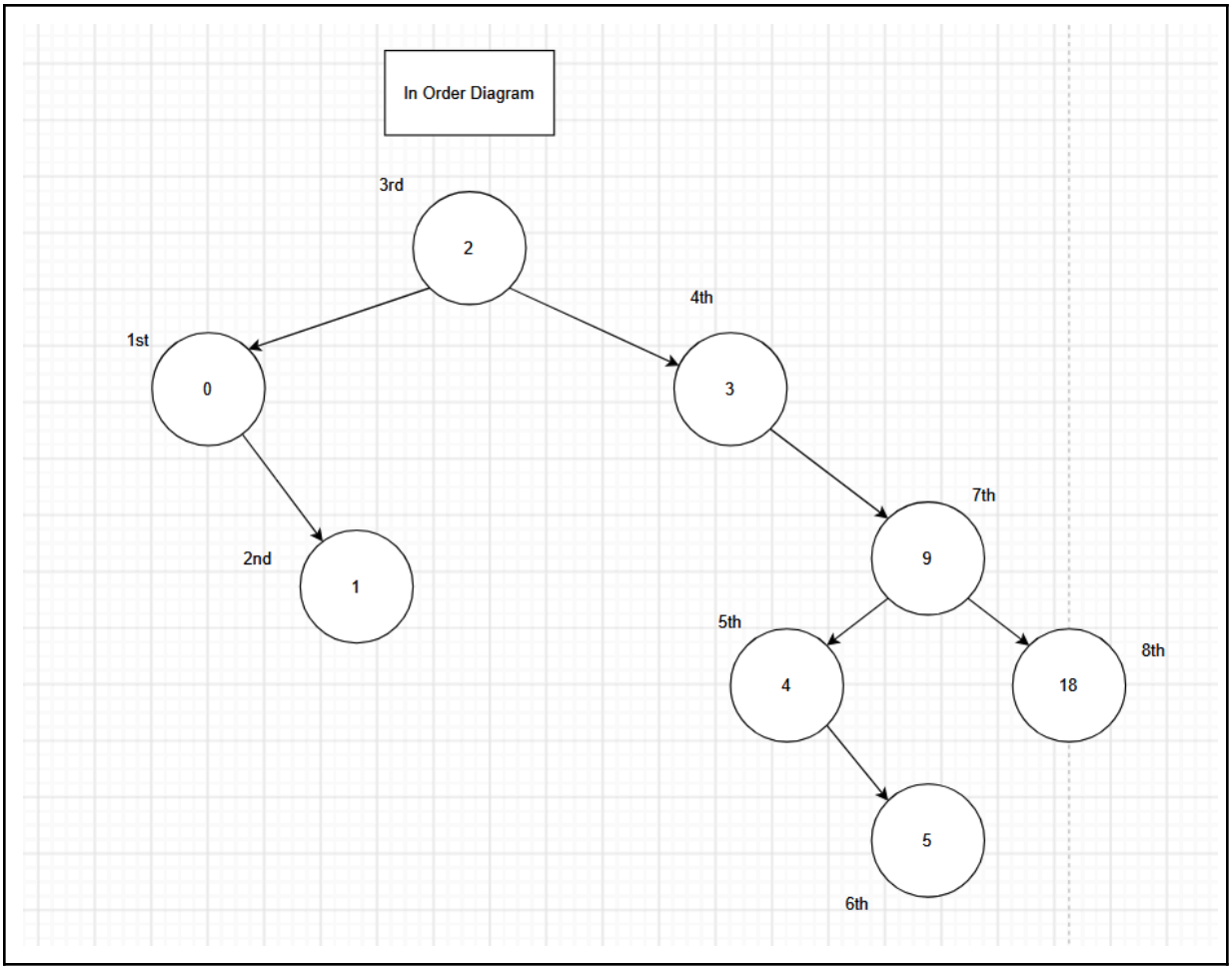
```
Binary Search Tree constructed and populated successfully.
Inserted sequence: 2, 3, 9, 18, 0, 1, 4, 5
```

Step 2:

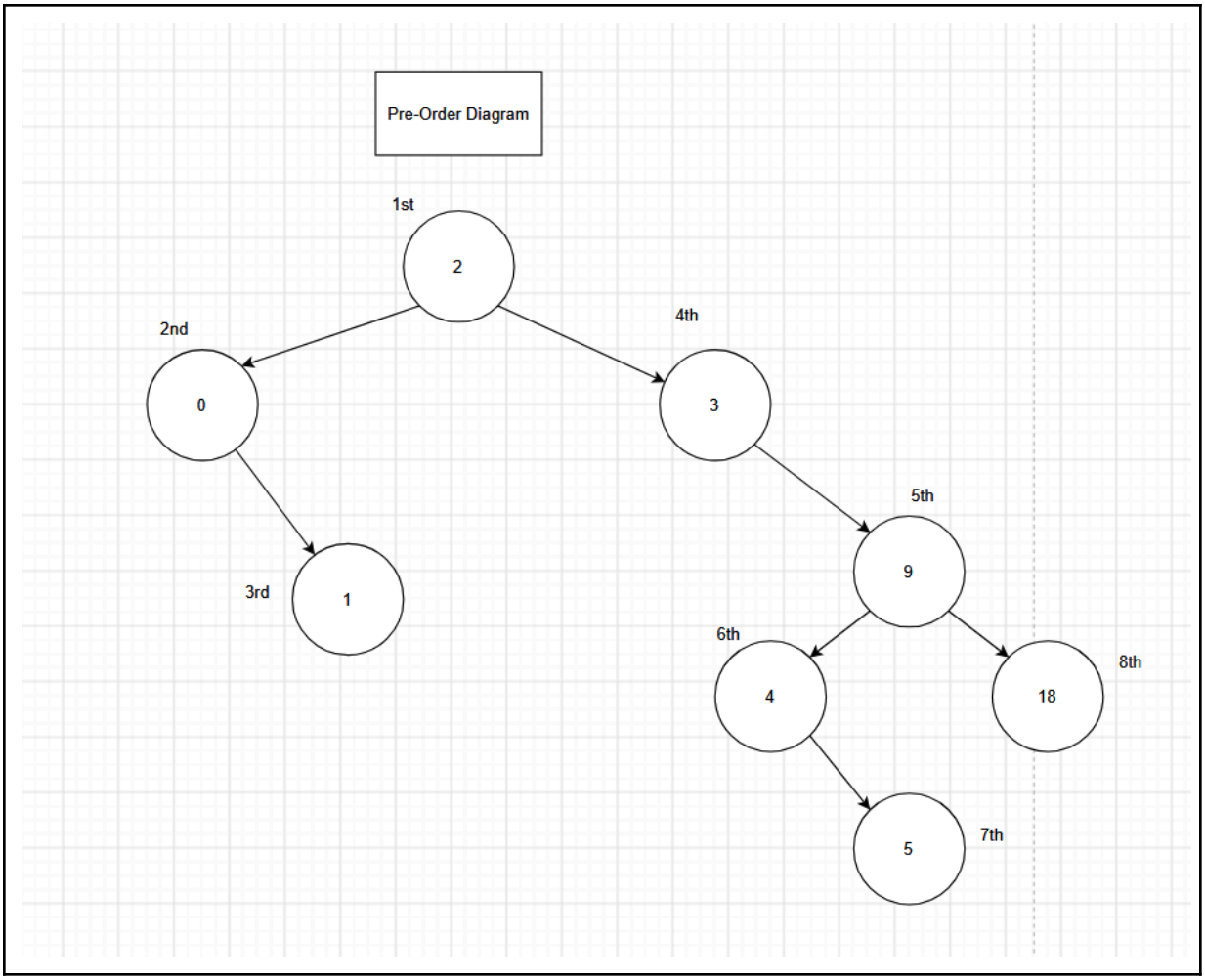
Base Diagram



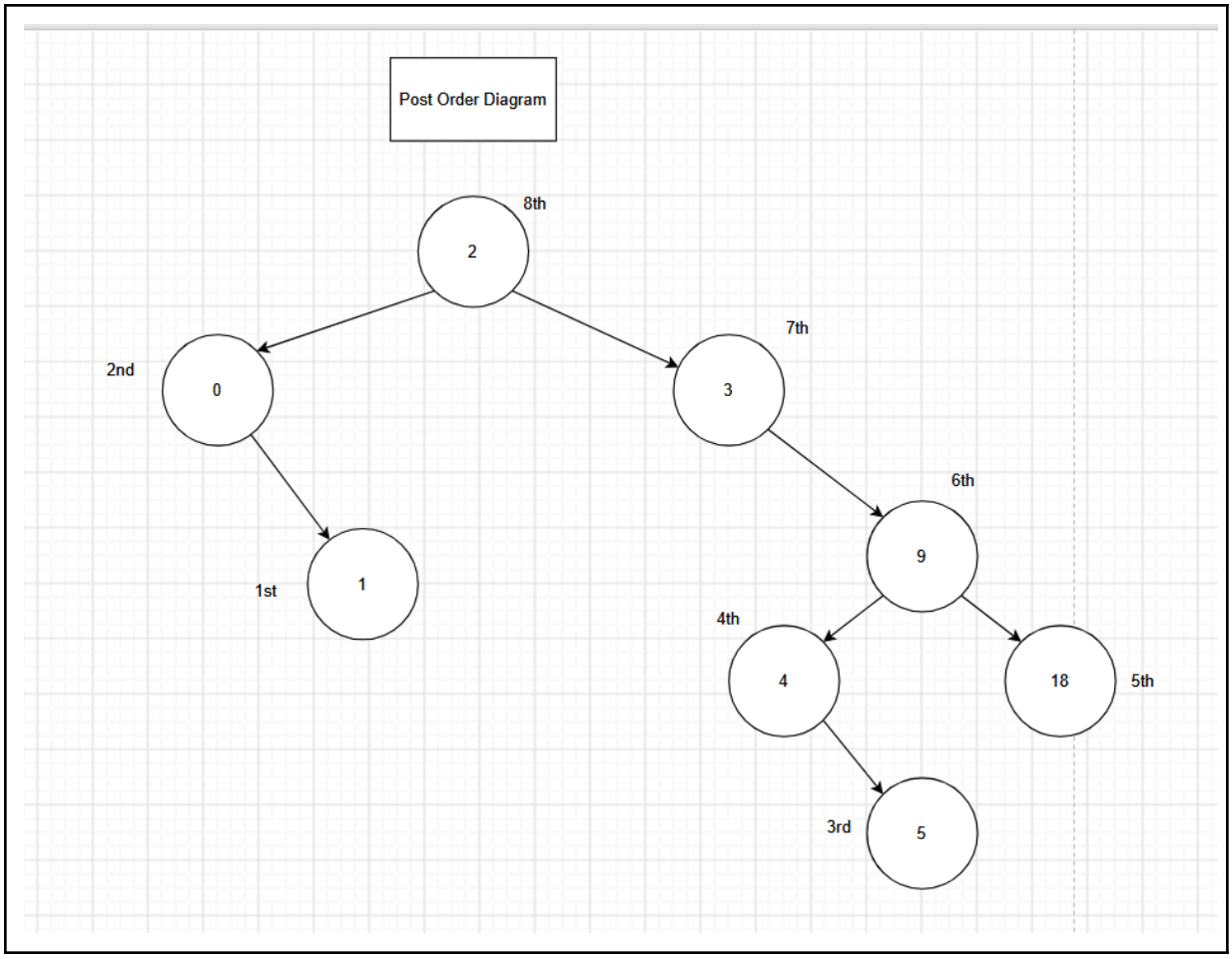
In Order Diagram



Pre-Order Diagram



Post Order Diagram



Step 3:

```cpp
template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << Key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::inOrder() {
    if (left != nullptr) {
        left->inOrder();
    }
    std::cout << Key << " ";
    if (right != nullptr) {
        right->inOrder();
    }
}


template <typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << Key << " ";
```

Output:

```
In-order:   0 1 2 3 4 5 9 18
Pre-order:  2 0 1 3 9 4 5 18
Post-order: 1 0 5 4 18 9 3 2
```

Conclusion:
This lab activity successfully demonstrated the implementation and traversal of hierarchical tree data structures in C++ using raw pointers. By building a general tree through the First-Child / Next-Sibling model, arbitrary child nodes were attached and searched dynamically across multi-level hierarchies. Additionally, constructing a Binary Search Tree with the numerical sequence 2, 3, 9, 18, 0, 1, 4, 5 highlighted key-sorting rules, where executing an In-order traversal produced a strictly sorted ascending sequence. Ultimately, the experiment reinforced essential dynamic memory management, pointer manipulation, and recursive traversal algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.